QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 4
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 6
5. Test Case Specifications (Drafts) 7
6. Test Strategy & Plan Sign-Off 8

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - Team Hack(h)ers*
Team Repo URL	*https://github.com/aeeya/StockSage*
Project Board URL	*N/A*
Live Deployment URL (optional for preliminary round)	*http://localhost:8081*

Objective:

*The primary objective is to ensure that **StockSage** can reliably process inventory data, calculate profit at risk, and generate accurate **Z.AI Intelligent Action Plans** for SME stock management. The system must maintain high data integrity between the Oracle database and the dashboard UI, while ensuring low-latency API responses for real-time decision support under concurrent user traffic.*

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing back to specific user requirements via a Requirement Traceability Matrix. It ensures every test has been conducted to prevent bugs or failures for those specific requirements and avoids unplanned feature creep.

1.1 In-Scope Core Features

- **Inventory Dashboard:** Real-time visualization of metrics including Current STock Value, Profit at Risk, and Estimated Savings via Z.AI.
- **AI Strategy Generation:** Automated **Z.AI STRATEGY** recommendations that provide specific action plans and logic insights based on inventory health and environmental factors.
- **Inventory Health Monitoring:** Tracking product details including Name, SKU, Stock Levels, and Expiry Status with visual color-coded badges (Critical, Warning, Safe).
- **Data Integration**: Seamless data fetching between the Spring Boot backend (/api/dashboard-data) and the frontend dashboard.

1.2 Out-of-Scope

- **Manual stock entry:** The current system uses a mock data service and does not yet include a form for manual product additions.

© UMHackathon 2026

- **User Authentication/Profile Management:** The application currently loads a single dashboard view without a login or profile editing gate.
- **Payment Processing:** This system focuses on decision intelligence rather than transactional processing.

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
API Connectivity	*3*	*5*	*15 (High)*	*Implement ''Offline'' status indicators in UI*	*Perform Negative Testing by stopping the Spring Boot server and verifying the refreshDashboardData() catch block displays the "Z.AI Offline" message*
Database Sequence Mismatch	*2*	*4*	*8 (Medium)*	*Use @SequenceGenerator with explicit allocationSize in Product.java*	*Conduct Integration Testing on ProductRepository the to verify that new product IDs are generated correctly via the Oracle PRODUCT_SEQ without collisions*
Stale AI Recommendations	*2*	*3*	*6 (Medium)*	*Implement a Refresh Data trigger in the dashboard*	*Refresh Functional Testing by clicking the "Refresh Data"button*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
  - *Scope: AIService logic and Product model validation.*
  - *Execution: JUnit 5 and AssertJ frameworks integrated within the Spring Boot environment.*
- **Integration Test**
  - *Scope: TestController to AIService data flow.*
  - *Execution: Performed after verifying individual service methods, ensuring the REST endpoint /api/dashboard-data correctly aggregates inventory and strategy data.*
  - *Workflow: To ensure API calls and Oracle database interactions via ProductRepository are working; real calls are tested.*
  - *Pass Condition: When the backend successfully returns a structured JSON response.*
- **Test Environment (CI/CD Practice):**
  - *Local Testing: Manual verification of the dashboard UI at https://localhost:8081.*
  - *Staging/CI: GitHub Actions triggers automated Maven build and tests on every push to the repository.*
  - *CI/CD automated pipeline: Deployment checks ensure the PRODUCT_SEQ is correctly configured for the target environment.*

- **Regression Testing & Pass/Fail Rules:**
  - *Execution Phase: Re-testing features each time a new feature is merged to the main branch.*
  - *Pass/Fail Condition: Test cases pass only when the UI accurately reflects data from AIService.java (e.g., Susu Segar showing "45 units").*
  - *Continuation Rule: Critical logic in AIService must pass 100% before allowing UI-level testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Test Data Strategy:**
    - *Manual: Mock items like "Susu Segar 1L" and "Roti Gardenia Jumbo" defined in AIService.*
    - *Automated: SQL seed scripts used to populate the PRODUCTS table and initialize the PRODUCT_SEQ sequence.*

- **Passing Rate Threshold:**
    - *Condition: A minimum of 90% of overall test cases must pass. For critical AI decision-making logic and database connectivity, a 100% passing rate is required to prevent financial inaccuracies in production.*

4. CI/CD Release Thresholds & Automation Gates

This section defines the metrics and thresholds for success criteria. If a threshold is not met, the pipeline blocks the forward transition to ensure stability.

4.1 Integration Thresholds (Merging to Main)

The main branch must remain a stable source of truth, achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***0 Errors***	***Passed***
API Performance	***Response Time < 800ms***	***~450ms (Standard for local Spring Boot GET)***	***Passed***
Security	***API keys hidden***	***Clean (No keys found in AIService.java or TestController.java)***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***91%***	***Passed***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***78%***	***Failed***
Critical Bugs	***Zero P0/01 Bugs***	***2***	***Failed***
API Performance	***Response Time < 800ms***	***450ms***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)***: This test consists of non-feature-based tests but architectural tests.*

Test Case ID	**Test Type & Mapped Feature**	**Test Description**	**Test Steps**	**Expected Result**	**Actual Result**
TC-01	***Happy Case (Entire Flow):*** *Dashboard Load / End-to-End Data Flow*	*Verify that the refreshDashboardData( ) function successfully populates the "Inventory Health" table with backend data.*	*1. Open the StockSage web application at http://localhost :8081.* *2. Click the* ***"Refresh Data"*** *button to trigger the API*	*The table correctly displays mock items like "Susu Segar 1L", "Roti Gardenia Jumbo", and "Coklat Cadbury" with appropriate status badges*	*Successfully populated the table. All stock values and badges align with the AIService.java mock data.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			call. *3. Observe the population of the Inventory Health table.*	*(Critical/Warning /Safe) based on stock levels.*	*Status: Passed*
TC-02	***Specific Case (Negative):*** *Backend Offline / Error Handling*	*Verify the system handles a server disconnection gracefully without crashing or showing raw code errors.*	*1. Stop the Spring Boot backend server (StocksageApplication).* *2. Refresh the dashboard in the browser.* *3. Observe the Z.AI STRATEGY panel.*	*The UI displays "Z.AI Offline" in the strategy panel instead of remaining blank or showing a 404/500 error.*	*The backend didn't answer. While the app remains stable, a clearer user-facing error message is needed beyond the console log.* *Status: Failed*
TC-03	***NFR (Performance):*** *Performance / API Response Test*	*Ensure the dashboard data API remains responsive (< 800ms) under simulated concurrent usage.*	*1. Use Postman or JMeter to target the /api/dashboard-data endpoint.* *2. Execute 50 concurrent GET requests.* *3. Record the average response time.*	*Average response time should be < 800ms with a 0% error rate.*	*Average response time recorded as 1128ms (exceeding threshold), though error rate remained at 0%.* *Status: Failed*

6. AI Output & Boundary Testing (Drafts)

This section validates that the Z.AI integration produces acceptable decision intelligence and handles abnormal inputs gracefully.

6.1. **Prompt/Response Test Pairs**

These pairs test the logic used to generate the Z.AI STRATEGY panel in your dashboard.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Analyze inventory for Susu Segar (1L) with 45 units remaining.	Should identify high stock levels and suggest a promotion or bundling strategy to prevent waste.	Strategy: High stock detected. Action: Run a 'Buy 2 Save 10%' promotion.	Pass
AI-02	Generate action plan for Roti Gardenia Jumbo with expiry date: 2026-04-25.	Must recognize the upcoming expiry and prioritize immediate liquidation or donation.	Action: Immediate clearance sale (50% off) due to expiry in 2 days.	Pass
AI-03	Evaluate impact of heavy rain forecast on bread sales.	Should factor in environmental logic (weather) to adjust stock replenishment advice.	Environmental Logic: Rain detected. Suggesting 15% increase in bread stock.	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	50 unique SKU objects in a single JSON payload
Input used while testing	300 SKU objects
Expected Behavior	Reject the request with an error or implement chunking
Actual Behavior	System experienced high latency (~5s) but did not reject the input; no error message displayed
Status	Fail

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.3. Adversarial/Edge Prompt Test

- **Scenario**: Inputting a product with a negative stock count (e.g., -5 units).
- **GLM Response**: The AI attempted to enter a valid quantity.
- **System Handling**: Your backend needs a validation layer to prevent negative integers from reaching the AI service.

6.4. Hallucinating Handling

To mitigate hallucinations, StockSage uses Rule-Based Guardrails:

- The AI is restricted to a pre-defined set of actions in the AIService.java.
- The system cross-references AI suggestions with the Profit at Risk calculation to ensure financial advice stays within realistic bounds.

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my